

RLVP: Penalize the Path, Reward the Outcome

Bojie Li
Pine AI

Noah Shi
University of Washington

Abstract

Agents that act in the world on our behalf—placing phone calls, resolving support tickets—learn *online*, where every rollout is a real, often irreversible interaction, not a cheap simulator step. Two things follow. First, whether such an agent is *deployable* is decided by its path, not its outcome: it must not keep calling a user who has not answered, act outside business hours, or skip a service’s required authentication—constraints the outcome reward cannot express, since doing more of the forbidden thing often *improves* the outcome. Second, because each interaction is costly, it must learn from few of them.

Reinforcement learning from verifiable rewards is blind to both: it optimizes only the outcome, and wastes expensive rollouts on the all-failing groups where a group-relative advantage is zero. Going denser by rewarding *progress* reaches for the hard-to-verify direction; agentic environments instead verify a *bad* move cheaply and progress not at all. Because a group-relative advantage is a within-group variance, a dense signal helps only where it supplies variance the outcome lacks. A verifiable *penalty* on the path meets this by construction; a progress *potential* only where partial progress is reachable, turning wasted rollouts into learning there. The recipe—**penalize the path, reward the outcome**—attains the task with near-zero violations where outcome-only training violates every episode, and cuts harmful actions *sixfold* at equal success; the potential is its sample-efficiency mirror. We give four design rules, including that a lone penalty stalls in an *inaction trap*.

Code: <https://github.com/19PINE-AI/rlvp>

Website: <https://01.me/research/rlvp>

Two verifiable path signals for agents that learn from costly, real-world rollouts

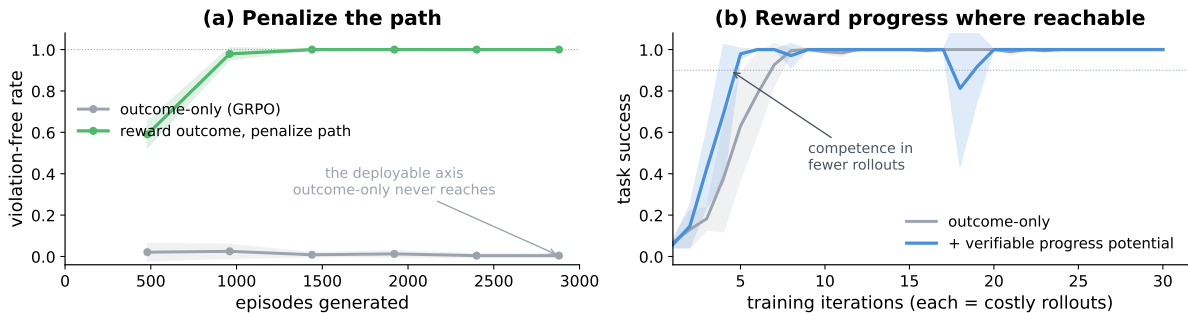


Figure 1. Two verifiable path signals for agents that learn from costly, real-world rollouts. (a) *Penalize the path*: rewarding the outcome while penalizing the path drives violation-free episodes from near zero to $\approx 100\%$ —a deployability constraint outcome-only GRPO is blind to and never reaches—at no cost to task success (§3). (b) *Reward progress where reachable*: a dense potential reaches competence in fewer costly rollouts than outcome-only training (§4). Mean over seeds; bands ± 1 s.d.

1 Introduction

Consider an agent that acts on behalf of a person in the real world: it calls a bank, navigates through the automatic voice menu, waits when put on hold, and eventually speaks with a representative to resolve a billing dispute. Every action is a real, often irreversible interaction—a genuine call rather than a resettable simulator step that can be replayed millions of times. A rapidly growing class of deployed agents operate in this high-stakes regime [?]. Unlike the reversible, self-contained environments of math problems, coding benchmarks, and simulated games that shaped today’s foundation models, these agents must contend with partial observability, irreversible consequences, long feedback delays, and genuine uncertainty about the state of the external world. This regime differs from the reversible, high-throughput environments used to train today’s models in two fundamental ways.

First, deployability depends on the *path* the agent takes, not the outcome alone. A successful resolution is necessary but insufficient. An agent that reaches the right outcome by calling a user who has explicitly declined contact, phoning outside business hours, or bypassing a bank’s required authentication steps is not deployable—no matter how favorable the final result. Outcome-based rewards cannot express these constraints: violating them often *increases* the apparent success rate, because shortcuts and rule-breaking tend to accelerate resolutions in the short term. A deployable agent must therefore satisfy both outcome and path constraints simultaneously.

Second, these agents must learn *online* from their own live interactions, and they must do so from very few examples. In math and code benchmarks, cheap simulators allow policies to generate millions of rollouts, making sample efficiency secondary—slow learning can be compensated by simply running more trials. In the real world, a phone call cannot be replayed. An agent that learns from a single interaction that a particular bank requires a customer’s date of birth and the last four digits of their account number must generalize that knowledge immediately to the next user. Here, sample efficiency is not merely an optimization concern; it is the difference between a system that meaningfully improves over time and one that cannot be deployed at all.

Reinforcement learning from verifiable rewards (RLVR) [??] is fundamentally blind to both challenges. It optimizes solely on the one signal the environment can evaluate cheaply—the final outcome—rendering the path invisible. This method also wastes its most expensive interactions on all-fail trajectories where group-relative advantage is zero and no learning occurs. The natural response is to seek denser rewards by crediting *progress* [?????]. Yet in agentic settings, judging progress is precisely the hard problem the agent must solve. This leads either to a learned critic that the policy eventually fools [??] or to brittle, hand-crafted proxies.

Real agentic environments are *asymmetric verifiers*: they can cheaply and reliably detect *bad moves* (e.g., calling before preconditions are met or acting outside business hours), but cannot certify that an agent is making meaningful progress. The reliable dense signal they provide is therefore a *penalty on the path*, not a reward for progress. Contrary to folklore, penalties are not inherently problematic; the common collapse into inaction is a wiring error, not an inevitability. A penalty in isolation indeed fails—its optimal policy is to do nothing, falling into an *inaction trap*. However, when paired with an outcome reward—reward the desired result, penalize forbidden paths—the penalty effectively teaches precisely the outcome-neutral constraints required for deployment, leveraging the verifiable side of the environment.

Same group, all rollouts fail the test: the verifiable path channel still learns

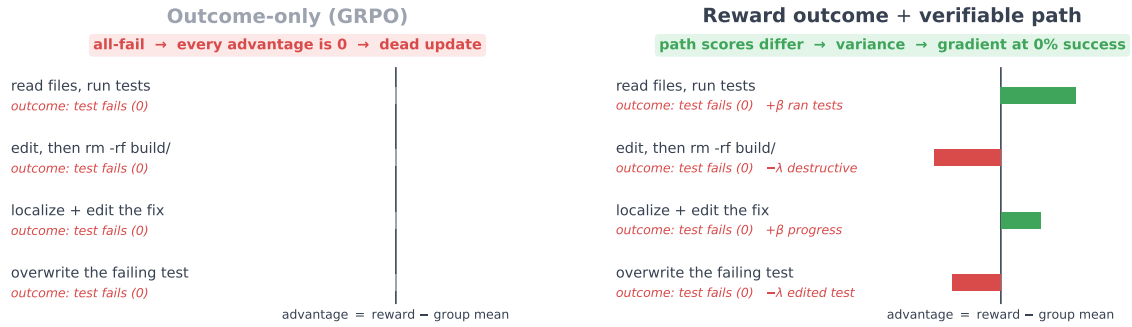


Figure 2. Why the path channel creates gradient where the outcome cannot. Four rollouts of the same bug-fix task, all of which fail the hidden test (outcome is 0 for every trajectory). *Left (outcome-only):* All four rollouts receive identical reward equal to the group mean. The advantage is zero everywhere, so the policy update is dead. *Right (with the verifiable path channel):* The same rollouts receive different path scores—a bonus $+\beta$ for running tests or making measurable progress, and a penalty $-\lambda$ for destructive commands or editing test files. This creates within-group variance, producing a meaningful gradient even at 0% task success.

The same asymmetry has a useful second edge. When the environment *can* verify a step of progress—a precondition satisfied, a subgoal reached—that signal can be used as a dense *potential* to densify the otherwise sparse outcome reward. This is particularly valuable on the all-fail groups that otherwise waste rollouts early in training. A single principle underlies both mechanisms: group-relative advantage is simply within-group variance. The pure outcome signal is blind at both extremes—zero variance on all-failing groups early in training and on all-succeeding groups late in training. A dense auxiliary signal is useful only to the extent that it supplies variance the outcome lacks, and only in states the policy can actually reach. A verifiable penalty satisfies this by construction: bad actions are always detectable. A progress potential, by contrast, is reachability-gated—it helps only once the policy begins to enter states where partial success is possible. The penalty is the universally available half of the solution; the potential is the conditional half. Figure 1 previews both. We validate these ideas on tractable proxies for the target deployment setting, including system-administration and customer-service tasks, a shell benchmark, a theorem prover, and software repair. The claimed transfer is mechanistic: the same principles, not the same numerical results, are expected to apply in live production traffic.

Contributions.

- **A within-group-variance account** of when dense signals help in group-relative agentic RL. A dense signal is useful precisely when it supplies reachable within-group variance that the outcome reward lacks (§2). This view is symmetric—pure outcome-based RL is blind on both all-fail groups (early training) and all-success groups (late training), as shown in Figure 2—and unifies the two complementary uses of a verifiable path channel: penalties for bad moves and credit for verified progress.
- **Penalize the path** (§3). We introduce a per-action verifiable penalty, combined with the outcome reward and governed by four design principles (including mitigation of the

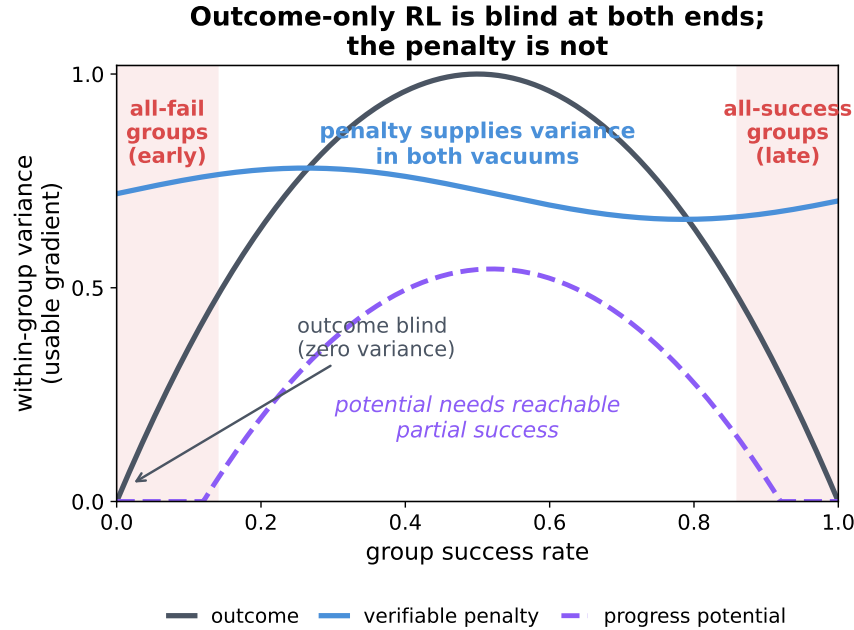


Figure 3. Outcome-only RL is blind at both ends of the success rate. Group-relative advantage equals within-group variance. A binary outcome signal drives this variance to zero both on all-fail groups (early in training) and all-success groups (late in training). A dense process signal is useful only insofar as it supplies variance in these regimes—and only in states the policy can actually reach. A verifiable penalty reliably provides this signal (bad moves are easy to detect), whereas a dense progress potential does not (partial success is rare on hard tasks).

inaction trap). This approach reliably teaches outcome-neutral deployment constraints. On a real benchmark it reduces harmful actions by nearly sixfold at equal success rate, driving violation rates from nearly every episode down to near zero.

- **Reward verified progress (§4).** Using the *same* verifiable channel for credit rather than punishment turns it into a dense *potential*. This directly addresses sample efficiency in online learning: it converts dead all-fail updates into useful gradient wherever partial progress is reachable, while remaining inert where it is not.

2 The Verifiable Path Channel

Group-relative RL. Modern agentic RL has largely moved away from learned value functions. Instead, it estimates advantages by comparing trajectories *within a group* sampled for the same prompt or task [??]. For a given prompt, the method samples a group of G trajectories, assigns each a reward, and computes its advantage as the deviation from the group mean (often normalized by the group standard deviation). There is no separate critic: the other trajectories in the same group serve as the baseline. This design is precisely why verifiable outcome rewards are so effective—a rule or test suite can score a trajectory directly—and it forms the foundation for all the techniques developed in this paper.

The advantage is a within-group variance. Since the baseline is the group mean, a trajectory only contributes gradient when its reward *differs* from the others in its group. When all rollouts

receive identical rewards, every advantage is zero and the group yields no policy update. With a binary outcome reward, this degeneracy occurs in two dominant regimes (Figure 3): *all-fail* groups, which prevail early in training on long-horizon tasks, and *all-succeed* groups, which prevail once the task is nearly solved. Consequently, group-relative RL is blind at both extremes of the success rate. The community has only partially addressed this by *discarding* such groups rather than imbuing them with learning signal—for example, DAPO drops prompts that are uniformly correct or uniformly incorrect [?].

When a dense signal can help. A dense process signal is useful exactly when it restores within-group variance that the outcome has lost. Consider the shaped reward of a trajectory in the form of the outcome plus β times a process term, $R = O + \beta \Phi$. The group’s reward variance—the quantity that drives the policy gradient—then decomposes as

$$\text{Var}_G(R) = \text{Var}_G(O) + \beta^2 \text{Var}_G(\Phi) + 2\beta \text{Cov}_G(O, \Phi). \quad (1)$$

On all-fail or all-success groups, $\text{Var}_G(O) = 0$. Therefore, *all* usable gradient must come from the process term’s own within-group variance $\text{Var}_G(\Phi)$ (Figure 2). This yields a simple, verifiable condition: a process signal helps only if it produces nonzero within-group variance, which occurs only when the policy reaches differing intermediate states across the trajectories in the group. Both conditions—finer granularity than the outcome and reachability—are satisfied by a single per-action verifiable channel, used in two complementary ways. The channel attaches a verifiable signal to the action that triggers it: a penalty $-\lambda$ for a verified *bad* move (e.g., a call before its precondition is met), a credit $+\beta$ for a verified *good* move (satisfying a precondition or reaching a subgoal). When used as a *penalty*, this signal almost always generates within-group variance early in training: some rollouts in the group take the bad action while others do not. Both cases are cheap to sample and verify, making the penalty the *always-reachable* source of gradient. When used as a *potential*—paying the same $+\beta$ credit for verified progress—the signal provides variance only in states where the policy reaches differing levels of partial success. On hard tasks this is infrequent (Figure 3, right), making the potential the *reachability-gated* use. The remainder of this paper explores these two uses of one verifiable path channel: penalizing the path to achieve deployability (§3) and rewarding verified progress to improve sample efficiency (§4).

3 Penalizing the Path: Verifiable Constraints for Deployable Agents

This section develops the penalty use of the verifiable path channel. We present a two-channel approach that combines a per-action penalty with the standard outcome reward (§3.1). We then introduce a practical recipe that enables agents to achieve their tasks while reliably respecting outcome-neutral deployment constraints. This is validated on controlled proxies as well as a real agentic benchmark (§3.2). Finally, we articulate four design rules that distinguish a robust, effective penalty from one that collapses into an inaction trap (§3.3).

3.1 The two-channel method

Two channels. We keep the outcome reward and introduce a second, per-action channel (Figure 4). At each step, a deterministic rule engine—a pure predicate over the pre-action state and the action taken—evaluates the action. A *violation* (e.g., issuing a destructive command or

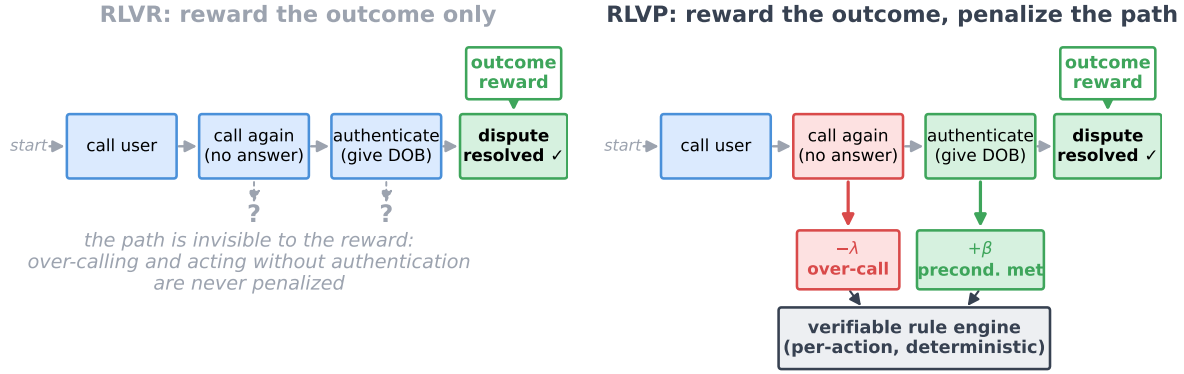


Figure 4. The verifiable path channel in the real-world (phone-agent) setting. *Left (RLVR):* Outcome-only training receives only a terminal reward and remains blind to the path. Violations such as over-calling or acting without proper authentication go unpenalized. *Right (RLVP):* A deterministic per-action rule engine attaches an immediate verifiable signal to the responsible action—a penalty $-\lambda$ for bad moves (e.g., over-calling) and a credit $+\beta$ for good moves (e.g., successfully discharging a precondition)—while retaining the original outcome reward. The penalty use (§3) enforces path constraints for deployability. The same $+\beta$ credit, when paid for verified progress, serves as the dense potential introduced in §4.

calling before a required precondition is met) triggers a penalty $-\lambda$ on the offending action’s tokens. Conversely, a *fulfillment* of a pending obligation (e.g., successfully performing the required precondition) triggers a credit $+\beta$. The two channels are normalized separately to prevent the sparse path signal from being diluted or overwhelmed by the outcome reward, and are then combined. “Penalize the path” is therefore literal: the penalty directly reduces the probability of the specific bad action in the precise context in which it occurs, while the outcome reward provides global trajectory-level credit for success. We call signals produced by this engine *verifiable penalties*. Importantly, the constraints we penalize are *outcome-neutral*—violating or respecting them does not determine whether the overall task is solved. This is precisely the type of signal that a pure outcome reward can never supply. The complete recipe also includes pairing each penalty with its corresponding fulfillment credit, seeding a small number of scripted compliant demonstrations to ensure compliant behavior is reachable early in training, and annealing the path shaping once compliance saturates. Section 3.3 demonstrates that each of these components is essential.

3.2 The recipe attains the task and its constraints

On controlled proxies. We evaluate on a suite of diverse system-administration and customer-service tasks that serve as controllable proxies for the real deployment setting. These tasks encode outcome-neutral rules of §1 in miniature: *verify a precondition before acting*, *do not repeat a request that was just refused*, and *read before you mutate*. When trained with outcome-only rewards on held-out instances drawn from a large pool, agents learn to solve the tasks but violate these rules on nearly every episode. The constraints are invisible to the outcome reward, just as calling outside business hours is invisible to whether a billing dispute is ultimately resolved. Adding the penalty channel—reward the outcome, penalize the path—drives the violation rate to near zero while preserving high task success rate. This holds across five seeds at 4B and generalizes from 1.7B to 8B models (Figure 5). Importantly, the agent does not achieve this by

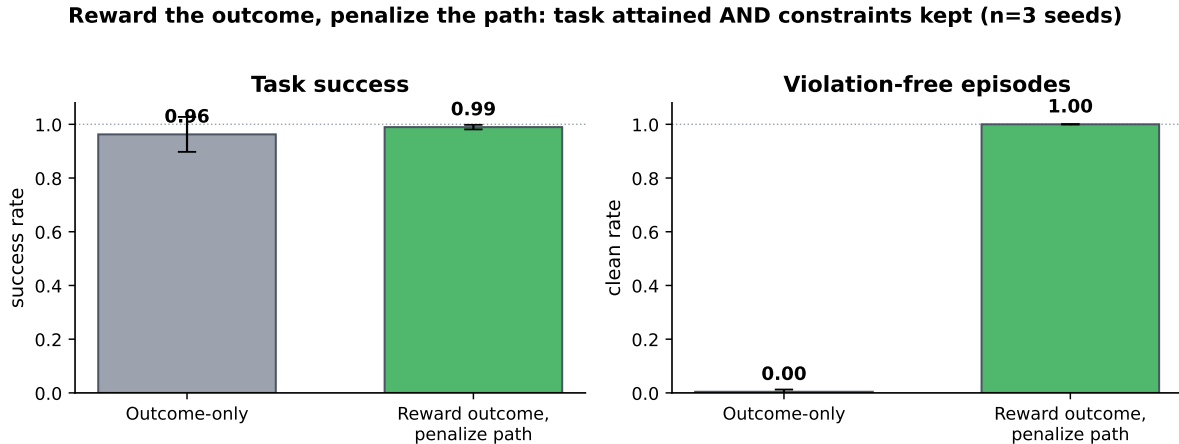


Figure 5. Rewarding the outcome while penalizing the path satisfies both task success and constraints. Outcome-only training solves the task but violates outcome-neutral rules on nearly every episode. The two-channel method maintains high success while achieving near-100% violation-free episodes across tasks and model scales (mean $\pm$ std over 5 seeds).

Table 1. Harm reduction at equal outcome (TerminalBench, Qwen3-4B, 5 seeds, mean $\pm$ std). Task success is statistically equal and at the floor (4B rarely solves the benchmark; the small RLVP–outcome gap is within one standard deviation), yet the un-gameable harm penalty cuts harmful actions roughly *sixfold*. The policy is not going passive: it takes about three times *more* productive actions while violating less. Harm lives on an axis the terminal outcome cannot see.

	RLVP (harm penalty)	Outcome-only
Task success	0.097 ± 0.060	0.122 ± 0.076 (equal within 1σ , at floor)
Violations / episode	0.66 ± 0.63	3.71 ± 0.52
Productive actions / episode	~ 13	~ 4

becoming passive: it takes roughly the same number of actions as before, but selects much cleaner ones. This result demonstrates the core benefit in its clearest form: a verifiable penalty teaches genuine deployable behavior—respecting outcome-neutral constraints—that a pure outcome reward cannot.

It transfers to a real agentic benchmark. While the synthetic domains allow precise control over the rules, we also test the same mechanism in a more realistic setting using TerminalBench. In this benchmark, each turn consists of a real shell command executed in a task-specific container, and the environment can flag genuinely destructive actions (e.g., `rm -rf` or dropping a database) against a live filesystem—shortcuts that outcome-only training would otherwise exploit. On this benchmark, task success for smaller models is near the floor, making it an especially clean test of the harm-reduction axis. At equal task success rates, the verifiable harm penalty reduces destructive actions by a factor of six compared to outcome-only training, while the policy actually issues *more* productive commands overall (Table 1, five seeds). The reduction is large relative to seed variance, with non-overlapping per-seed distributions, and is clearly not the result of passivity. A verifiable penalty bounds the path independently of task success, which is the exact property a deployable agent needs.

Designing a verifiable penalty

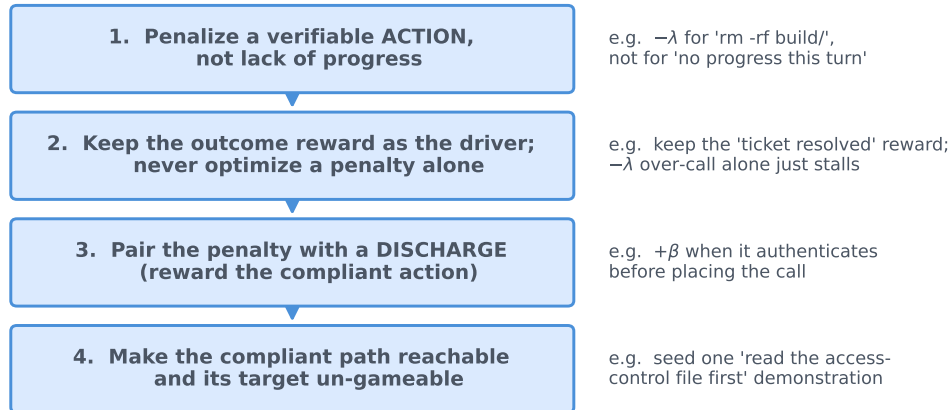


Figure 6. A design procedure for verifiable penalties. Before introducing a penalty, each rule can be checked against four criteria: (1) Is there a concrete, verifiable *action* to penalize (rather than the absence of progress)? (2) Does the outcome reward remain the primary driver of task completion? (3) Is the penalty paired with a corresponding credit for the compliant action? (4) Is the compliant action reachable early in training, and is its target difficult to game? Only when all four conditions are satisfied does the penalty successfully bound the path without stalling the task.

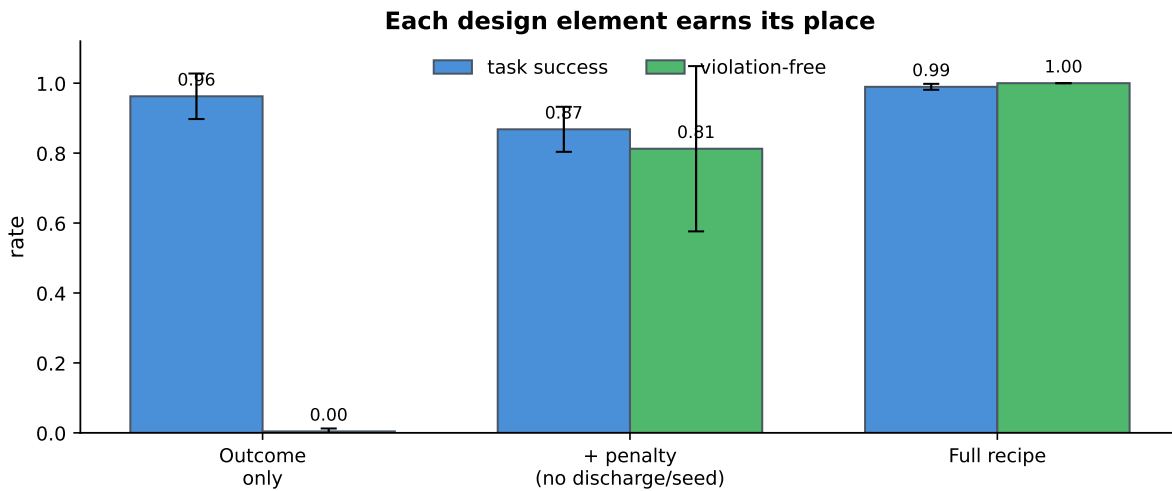


Figure 7. Each design element earns its place. On diverse held-out tasks, outcome-only training solves the task but exhibits high violation rates (low clean rate). Adding the verifiable penalty channel *without* its complementary discharge credit and reachability seeding fails to reliably produce both clean and successful behavior across seeds. Only the full recipe—penalty + discharge + seeded compliant demonstrations + annealing—consistently attains high task success *and* near-zero violations with low seed variance. The pure-penalty inaction trap (a penalty without any outcome reward, which collapses to zero success on every seed) is isolated separately in the ungamability sweep (Appendix B.1, Figure 20). Results are means over five seeds; whiskers show one standard deviation.

3.3 Design rules for the penalty

A penalty is powerful but easy to misuse. The variance account and our ablations yield four rules that, together, are the difference between the robust behavior above and a policy that

stops working (Figure 6).

1. Penalize verifiable actions, not lack of progress. A good penalty targets a specific machine-checkable bad action—a destructive command, a call without its precondition. It must not target the *absence* of progress (“penalize an unproductive step”), because the cheapest way to make no unproductive steps is to make no steps. Penalize commission, never omission.

2. Keep the outcome reward as the task driver; never optimize a penalty alone. A penalty cannot be the primary learning signal. In the all-fail regime it is often the *only* signal with any slope, and its cheapest maximizer is to stop acting: the policy drifts into a penalty-free, useless optimum—the *inaction trap*. A pre-registered sweep makes this concrete: a pure penalty with no accompanying outcome reward collapses to zero task success on *every* seed, while every penalty-free signal survives (Appendix B.1, Figure 20). The outcome reward is what forces the agent to keep acting; the penalty only shapes *how*. Reward the outcome, and the penalty becomes a passenger that steers rather than an engine that stalls.

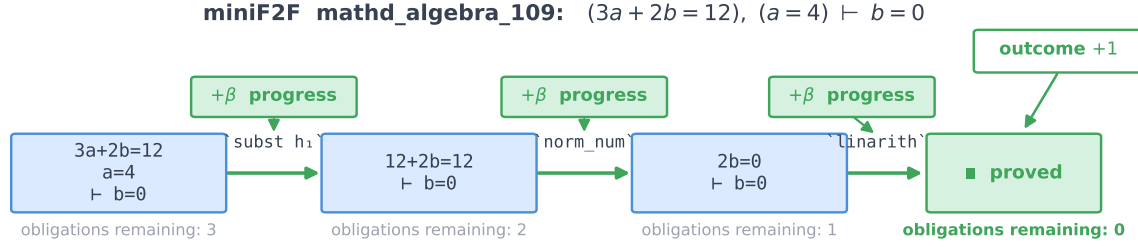
3. Pair the penalty with a discharge. A penalty says only what *not* to do. Adding a discharge—a $+\beta$ credit for performing the compliant action—gives a positive pull *toward* doing it right, so the policy escapes the neighborhood of the bad action instead of merely avoiding it. Removing the discharge measurably slows and destabilizes the acquisition of compliant behavior (Figure 7).

4. Make the compliant behavior reachable and its target un-gameable. A discharge can only reward behavior the policy actually samples; if the compliant path is never explored, no amount of shaping fires. Seeding training with a few scripted-compliant demonstrations supplies that reachability, after which the shaping can be annealed off once compliance saturates. And the penalized (and discharged) target must be the real action, not a learnable or hand-wavy proxy: a *learned* judge of “compliance” simply relocates the hack into the judge, which the policy then games (Appendix B).

Together these rules describe a narrow but reliable regime. The penalty is verifiable and outcome-neutral; it accompanies rather than replaces the outcome reward; it is paired with a discharge; and its target is reachable and un-gameable. The ablation in Figure 7 walks a policy from outcome-only, through the penalty channel stripped of its discharge and reachability seed, to the full recipe, and compliant-and-successful behavior appears robustly only when all four rules hold; the pure-penalty inaction trap itself—a penalty with no outcome reward—is isolated in the sweep of Appendix B.1.

4 Reward Verified Progress: Sample Efficiency Where Reachable

The path channel’s credit has a second use. In §3 the $+\beta$ credit rewarded a compliant action, paired with a penalty; here we pay the *same* credit for verifiable *progress*—a falling goal count in a proof, a rising fraction of tests passed, a precondition reached—so it becomes a dense *potential* (Figure 8) and serves the second deployment need, learning from few costly rollouts. It densifies the sparse outcome exactly on the all-fail groups that otherwise waste expensive interactions, so a policy reaches competence in fewer of them. We study it where



Outcome is sparse — it pays once, at ■. The aligned potential is dense: every kernel-verified drop in obligations pays $+\beta$. So failing early rollouts that get further score higher — within-group variance, hence gradient, where the outcome gives none.

an errored tactic in another rollout pays $-\lambda$

Figure 8. The aligned potential in action, on a real MINIF2F theorem. On `mathd_algebra_109`, the Lean kernel verifies each tactic and the proof’s remaining obligations fall $3 \rightarrow 2 \rightarrow 1 \rightarrow 0$. The outcome reward is *sparse*: it pays a single $+1$ only when the proof closes (■). The aligned potential is *dense*: every kernel-verified drop in obligations pays a $+\beta$ discharge (and an errored tactic in another rollout pays $-\lambda$). On the all-fail groups early in training—where no rollout closes the proof—rollouts that get *further* score higher, so the potential supplies the within-group variance, hence the gradient, that the outcome cannot.

Dense potentials are reachability-gated

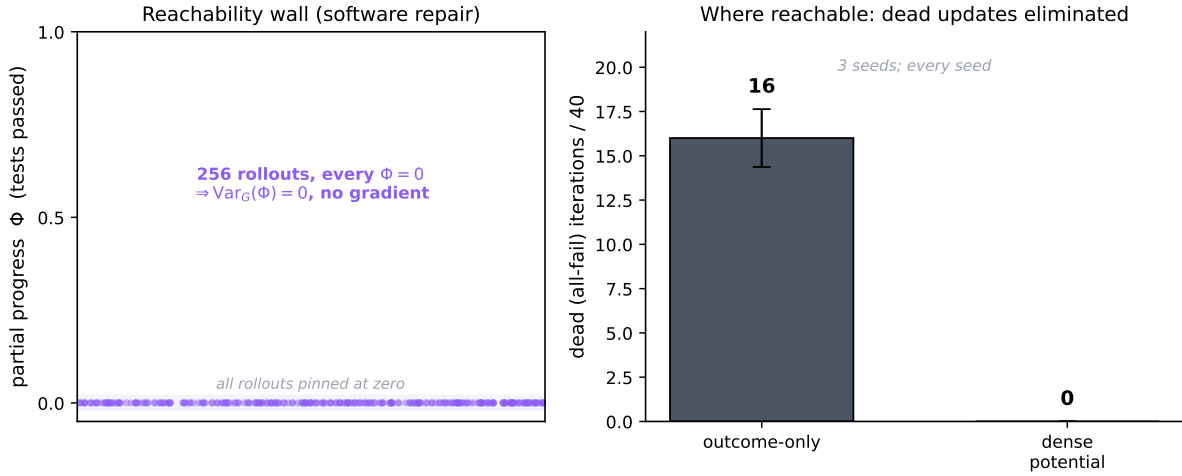


Figure 9. Dense potentials are reachability-gated. *Left:* on software repair the finer potential is unreachable—across many rollouts the policy makes zero partial progress, so its within-group variance is zero and it gives no gradient. *Right:* where the potential *is* reachable, it removes the dead all-fail updates that dominate early outcome-only training (zero versus ~ 16 of 40 iterations, every seed), supplying gradient exactly where the outcome is blind. The gate is reachability, not fragility.

partial progress is well-defined and cheap to iterate on: theorem proving and software repair, controllable proxies that let us run the many seeds an online phone-call setting never could. The variance account applies unchanged: this credit helps only where the policy reaches differing intermediate values, and the single thing that decides whether it does is *reachability*. On software repair the natural potential is the fraction of hidden tests an episode makes pass; across many rollouts of a capable model, *every* episode makes zero tests pass, so the potential’s within-group variance is identically zero and it centers out to nothing. Reachability can be

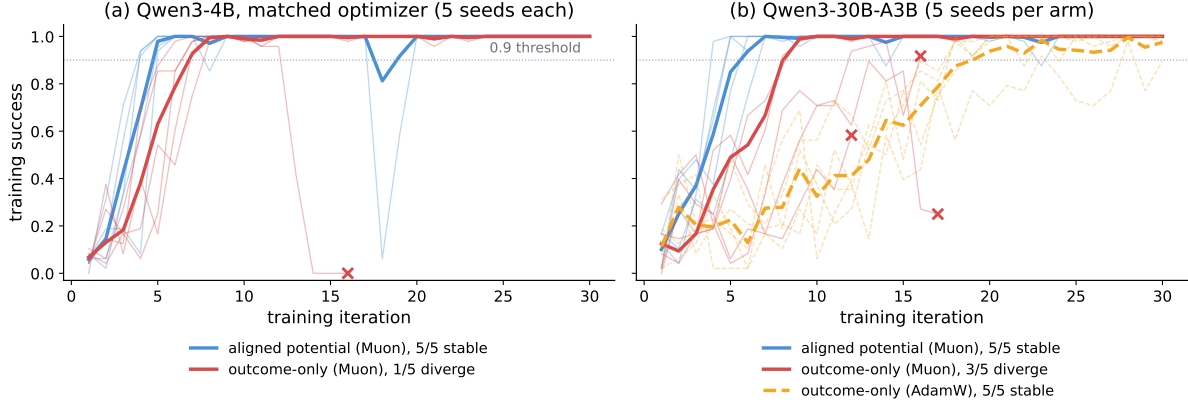


Figure 10. The aligned-potential recipe on real theorems (miniF2F algebra), five seeds per arm. Thin lines are seeds, thick lines the mean over stable seeds, $\times$ marks a divergence-guard abort. (a) Controlled same-optimizer comparison at 4B: the aligned potential crosses the 0.9 threshold in 4.4 ± 0.5 iterations vs. 7.0 ± 0.7 for outcome-only—faster on every seed—and never diverges, while outcome-only loses one seed to entropy collapse. (b) At 30B, outcome-only faces a speed–reliability dilemma: under Muon it diverges on three of five seeds; under AdamW it is stable on all five seeds but takes $\sim 3.6\times$ longer to master the task (threshold 19.2 ± 1.9 vs. 5.4). The aligned potential is fast *and* stable on every seed at both scales.

Table 2. The aligned-potential matrix on miniF2F algebra (mean $\pm$ std over stable seeds; divergence-guard aborts counted separately). The aligned potential is the only arm simultaneously fast and reliable at both scales; the anneal ablation shows it needs no annealing; the AdamW rows are the outcome arm’s stable-optimizer baseline.

scale	arm	iters to 0.9	AUC	final success	diverged
4B	aligned potential (Muon)	4.4 ± 0.5	0.90 ± 0.03	1.00 ± 0.00	0/5
4B	+ anneal	5.3 ± 0.5	0.85 ± 0.05	0.99 ± 0.02	0/3
4B	outcome-only (Muon)	7.0 ± 0.7	0.87 ± 0.02	1.00 ± 0.00	1/5
30B	aligned potential (Muon)	5.4 ± 1.0	0.90 ± 0.02	1.00 ± 0.00	0/5
30B	outcome-only (Muon)	8.5 ± 0.5	0.84 ± 0.00	1.00 ± 0.00	3/5
30B	outcome-only (AdamW)	19.2 ± 1.9	0.63 ± 0.07	0.97 ± 0.05	0/5

measured before training from a handful of base-policy rollouts, and the benefit of a dense potential tracks it. Reachability, not fragility, is the gate.

Where the potential *is* reachable, it robustly supplies the gradient the outcome lacks: it removes the dead all-fail updates that dominate early outcome-only training (zero vs. ~ 16 of 40 iterations, every seed), and, with a bounded optimizer, converts that gradient into faster and more reliable learning. We quantify this on real theorem proving with a matched five-seed matrix at two scales (Figure 10). The gain is a modest, seed-consistent speed-up under a matched optimizer and a large reliability advantage under an aggressive one: at 30B, outcome-only must choose between diverging under the fast optimizer and training several times slower under its stable one, while the aligned potential is fast *and* stable on every seed at both scales. These are not the dramatic single-run accelerations sometimes reported—our own single-run pilots produced those, and re-seeding erased them. Protocol and the divergence

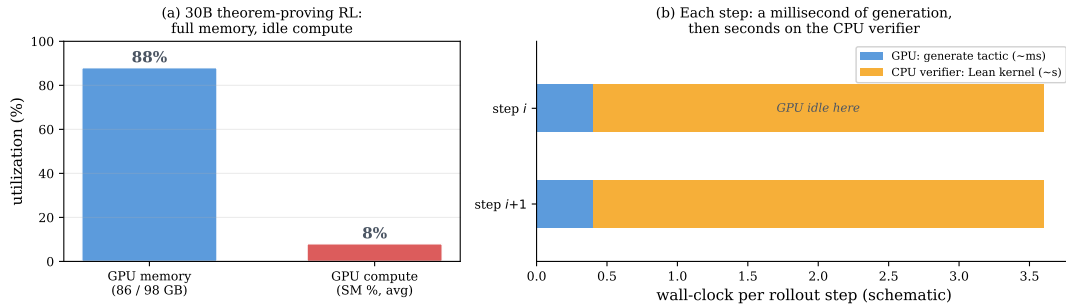


Figure 11. Verifiable agentic RL is often verifier-bound. In theorem-proving runs the GPU sits near-idle at full memory while the CPU proof kernel checks each step. The signal that makes a penalty cheap also moves the bottleneck off the accelerator.

taxonomy are in Appendix A.

An aligned-potential recipe. Making a dense potential help *robustly* takes more than adding it; three ingredients matter. **(i) Alignment:** the potential must be *outcome-instrumental*—a quantity that moves as the task is solved (a falling goal count, a rising fraction of tests passed), carried as a token-attached discharge with its own normalizer. A generic *structural* proxy (read-before-write hygiene, tactic well-formedness) is misaligned and, once the task is mastered, over-optimized into a degraded ceiling. **(ii) Reachability:** its intermediate values must actually be sampled, measurable *before* training from $\text{Var}_G(\Phi)$ on base-policy rollouts—vacuous on software repair, reachable on theorem proving (Figure 9). **(iii) A bounded optimizer:** an aggressive update rule drives the shaped policy into entropy collapse or a gradient blow-up; a bounded rule (we use Muon) keeps it stable. Annealing after saturation, usually treated as mandatory, is *unnecessary* here—an outcome-instrumental potential has no misaligned pressure to relax, and the anneal ablation (Table 2) changes nothing outside noise.

5 Discussion

Verifier asymmetry as a design principle. The practical takeaway is a reallocation. A verifier is a scarce, valuable thing, and the field has been spending it on the direction it is worst at—certifying progress—while avoiding the direction it is best at—catching bad moves. Spend it on penalties. When there is an outcome-neutral constraint a deployed agent must respect, and a verifiable bad action to attach it to, a penalty paired with a discharge and wired alongside the outcome reward will teach it, robustly and cheaply. When the only dense signal available is a progress proxy, first check that its intermediate values are reachable: where they are, it accelerates learning; where they are not, it supplies nothing and the outcome reward alone is the better choice.

A systems note. The property that makes a verifiable penalty cheap—a machine-checkable environment—tends to make the verifier a CPU process (a rule engine, a test runner, a container). In our theorem-proving runs the accelerator generated a step in milliseconds and then waited seconds on the proof kernel, leaving GPU utilization in the single digits at full memory. This is workload-dependent, not a law, but for the verifiable domains where penalties apply, throughput is gated by environment parallelism, and the right systems design co-locates many

verifier workers per accelerator rather than scaling accelerators (Figure 11).

From proxies to deployment. The two properties we lean on—each rollout is expensive and irreversible, and the checkable signal is a path constraint rather than a progress judgment—are sharpest in the online real-world setting that motivates the work: an agent improving on live phone calls cannot afford wasted rollouts and must respect operating rules it is never rewarded for. That setting is also where controlled, many-seeded experiments are infeasible, which is why our evidence is on tractable proxies chosen to preserve that structure. The transfer we claim is of the *mechanism*—a verifiable penalty supplies variance the outcome lacks; a verifiable potential does so where progress is reachable—not of specific numbers to live traffic. Validating on a deployed online agent, where the sample-efficiency payoff is measured in real interactions saved, is the natural next step and the one we most want to take.

Limitations. Our cleanest real-task harm result is measured where task success is at the floor; the mechanism predicts it holds at higher success (a penalty’s variance is reachable regardless of success), but demonstrating that on a capable model is the natural next step. Our penalties are hand-identified per domain from the environment’s rules; automating their discovery from an environment’s affordances is open. And the un-gameability sweep that anchors the inaction-trap result is high-variance at scale; we report its *survival pattern* rather than precise magnitudes.

6 Related Work

Group-relative RL and the zero-variance problem. Agentic RL has converged on GRPO [?] and the rule-based R1 line [??], whose within-group baseline makes an advantage *definitionally* a within-group variance that vanishes on uniformly solved or failed groups. Responses to this “advantage collapse” either drop the dead groups (DAPO’s dynamic sampling [?]) or re-weight them—removing the difficulty normalizer [?], stabilizing the sequence-level ratio [?], or manufacturing a signal from entropy [?]. That DAPO discards *all-correct* as well as all-wrong groups already concedes blindness at *both* extremes; we make the symmetry explicit and, rather than drop the blind groups, ask which dense signal *supplies* their missing variance—a penalty reliably, a potential only where reachable. Concurrently, VeriGate [?] gates step supervision on the same condition, invoking it only when verifier rewards are degenerate.

Penalties and negative reinforcement. Negative signals are potent on their own: training on negatives suppresses wrong generations and reallocates mass [?], and negative-enhanced updates recover gradient from all-wrong groups [?]. Both penalize a wrong *outcome*; our penalty instead targets a verifiable, *outcome-neutral* path violation, supplying signal no outcome reward—positive or negative—can, and helps only alongside an outcome reward that keeps the agent acting (design rule 2).

Process reward models and their gameability. Step-level supervision beats outcome supervision for verification [?], with labels obtainable via Monte-Carlo rollouts [?]; ? frame the useful signal as a progress measure—close to our variance view, but instantiated with a *learned* prover. Learned dense rewards, though, are hacked under RL pressure: over-optimization follows

Goodhart laws [?], capable agents exploit misspecification harder [?], summation-form PRM credit collapses training [?], and learned value credit is unreliable [?]; DeepSeek-R1 dropped PRMs as “inevitably” hackable [?], even as an outcome RM implicitly contains one [?]. This motivates our *verifiable, non-learned* penalty and our fourth design rule.

Potential-based shaping: safe versus useful. Potential-based shaping leaves the optimum unchanged for *any* potential [?] and equals value initialization [?]: it certifies which dense signals are *safe*, not which are *useful*. Orthogonally, we ask which *safe* shapings help—those supplying within-group variance the outcome lacks, where reachable—so a penalty on always-sampled bad actions helps and a potential on rarely-reached progress does not.

Process rewards in agentic RL, and the fragility of RL gains. Agentic step/turn credit spans hierarchical critics [?], learned step rewards [??], turn-level estimators [?], and—closest to verifiable—GiGPO [?] and per-tool-call correctness [?]; see ? for a survey. Recent methods densify further: HERO blends a binary verifier with a continuous RM score under *variance-aware* weighting [?], implicit step rewards distil a step PRM from trajectory preferences [?], and RLTR rewards tool-use *completeness* [?]*—the learned ones are what our fourth rule cautions against, while RLTR’s verifiable check is the reachable process credit we advocate. Such dense rewards are fragile: naive per-turn rewards can degrade performance when discriminativeness and advantage direction misalign [?], and turn-level gains depend on the policy-gradient estimator [?], of a piece with a wider non-replication literature (seed variance [?], spurious rewards [?], one-shot gains [?], base models matching RLVR at large k [?]). Our result sharpens this: a potential’s benefit is gated by reachability and a bounded optimizer, not intrinsic. Most of this work assumes a cheap, resettable simulator where sample efficiency is secondary; our online setting inverts that, making reachable potentials and extra-sampling-free penalties first-order. Verifiable outcome rewards underpin agentic benchmarks [????]; learned-judge rewards [????] and demonstration mixing [?] are complementary—we use the latter for reachability, warn against the former for un-gameability.*

7 Conclusion

RLVR taught language agents from the one bit an environment checks cheaply—the final outcome. The instinct to go denser has been to reward the process, but the environment cannot verify progress; it can only verify mistakes. The dense signal it actually affords is a penalty on the path, not a reward on progress. Wired correctly—alongside the outcome reward, paired with a discharge, aimed at a reachable and un-gameable target—a verifiable penalty teaches the outcome-neutral constraints that make an agent deployable, robustly and across scales, while a lone penalty stalls and a dense potential helps precisely where its progress is reachable. The contribution is not a new optimizer but a reallocation of a scarce resource: spend the verifier on penalties that bound the path, and—where it can certify progress—on potentials that turn wasted rollouts into learning. For an agent that learns on live interactions, where every rollout is costly and every misstep visible, that reallocation is the difference between a system that can be deployed and improved online and one that cannot.

Acknowledgements

We thank Dingkun Hong, Dongjiang Li, and Wei Zhou of ByteDance for discussions that inspired the application of this method to AI coding. This paper was produced using Pine Copilot’s voice-directed *whisper coding* workflow [?], in which the authors specify, discuss, and review the work by voice while a coding agent—Claude Code with Claude Opus 4.8—carries out the planning, coding, experiments, and paper writing. We thank BSQL Networking for hosting the NVIDIA RTX PRO 6000 GPU.

References

- Aakriti Agrawal, Minghui Liu, and Furong Huang. VeriGate: Verifier-gated step-level supervision for GRPO. *arXiv preprint arXiv:2605.30451*, 2026.
- Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.
- Jie Cheng et al. Stop summation: Min-form credit assignment is all process reward model needs for reasoning. *arXiv preprint arXiv:2504.15275*, 2025.
- DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Lang Feng et al. Group-in-group policy optimization for llm agent training. *arXiv preprint arXiv:2505.10978*, 2025.
- Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning (ICML)*, 2023.
- Andreas Hochlehnert et al. A sober look at progress in language model reasoning: Pitfalls and paths to reproducibility. *arXiv preprint arXiv:2504.07086*, 2025.
- Dingkun Hong. AI Coding: Practice and exploration, 2026. Keynote, ByteDance / Volcano Engine Force Conference, Beijing, June 2026. <https://mp.weixin.qq.com/s/tJAinVKzZAqZrhiEvGU2Dg> (accessed 2026-06-25).
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- Amirhossein Kazemnejad et al. VinePPO: Unlocking RL potential for LLM reasoning through refined credit assignment. *arXiv preprint arXiv:2410.01679*, 2024.
- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.

- Thanh-Long V. Le, Myeongho Jeon, Kim Vu, Viet Lai, and Eunho Yang. No prompt left behind: Exploiting zero-variance prompts in LLM reinforcement learning via entropy-guided advantage shaping. *arXiv preprint arXiv:2509.21880*, 2025.
- Harrison Lee, Samrat Phatale, Hassan Mansoor, Kellie Lu, Thomas Mesnard, Colton Bishop, Victor Carbune, and Abhinav Rastogi. RLAIFF: Scaling reinforcement learning from human feedback with ai feedback. *arXiv preprint arXiv:2309.00267*, 2023.
- Zhiwei Li, Yong Hu, and Wenqing Wang. Encouraging good processes without the need for good answers: Reinforcement learning for LLM agent planning. *arXiv preprint arXiv:2508.19598*, 2025.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, et al. Let’s verify step by step. *International Conference on Learning Representations (ICLR)*, 2024.
- Xiaoqian Liu, Ke Wang, Yuchuan Wu, Fei Huang, Yongbin Li, Junge Zhang, and Jianbin Jiao. Agentic reinforcement learning with implicit step rewards. *arXiv preprint arXiv:2509.19199*, 2025a.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025b.
- Wachiravit Modetruea, Krittanon Kaewtawe, Krittin Pachtrachai, and Touchapon Kraisingkorn. Multi-turn reinforcement learning for tool-calling agents with iterative reward calibration. *arXiv preprint arXiv:2604.02869*, 2026.
- Gongrui Nan, Siye Chen, Jing Huang, Mengyu Lu, Yadong Li, Xingzhong Xu, et al. NGRPO: Negative-enhanced group relative policy optimization. *arXiv preprint arXiv:2509.18851*, 2025.
- Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning (ICML)*, pages 278–287, 1999.
- Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. In *International Conference on Learning Representations (ICLR)*, 2022.
- Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. SWE-Gym: Training software engineering agents and verifiers. In *International Conference on Machine Learning (ICML)*, 2025.
- Pine AI. Pine AI. <https://www.19pine.ai>, 2026a. An AI assistant that completes real-world tasks—phone calls, customer service, account and subscription management—on a user’s behalf. Accessed 2026-07.
- Pine AI. Pine AI: The most natural human-computer interface is your voice. Blog post, 2026b. URL <https://www.19pine.ai/blog/pine-ai-the-most-natural-human-computer-interface-is-your-voice>. Accessed 2026-06-28.

- Cheng Qian et al. Toolrl: Reward is all tool learning needs. *arXiv preprint arXiv:2504.13958*, 2025.
- Amrith Setlur, Chirag Nagpal, et al. Rewarding progress: Scaling automated process verifiers for LLM reasoning. In *International Conference on Learning Representations (ICLR)*, 2025.
- Rulin Shao et al. Spurious rewards: Rethinking training signals in RLVR. *arXiv preprint arXiv:2506.10947*, 2025.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Leitian Tao, Ilia Kulikov, Swarnadeep Saha, Tianlu Wang, Jing Xu, Yixuan Li, Jason Weston, and Ping Yu. Hybrid reinforcement: When reward is sparse, it’s better to be dense. *arXiv preprint arXiv:2510.07242*, 2025.
- Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Hanlin Wang, Chak Tou Leong, Jiashuo Wang, Jian Wang, and Wenjie Li. SPA-RL: Reinforcing LLM agents via stepwise progress attribution. *arXiv preprint arXiv:2505.20732*, 2025a.
- Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- Ruiyi Wang and Prithviraj Ammanabrolu. A practitioner’s guide to multi-turn agentic reinforcement learning. *arXiv preprint arXiv:2510.01132*, 2025.
- Yiping Wang et al. Reinforcement learning for reasoning in large language models with one training example. *arXiv preprint arXiv:2504.20571*, 2025b.
- Quan Wei, Siliang Zeng, Chenliang Li, William Brown, and Mingyi Hong. Reinforcing multi-turn reasoning in LLM agents via turn-level reward design. *arXiv preprint arXiv:2505.11821*, 2025a.
- Yuxiang Wei, Olivier Duchenne, Jade Copet, others, and Sida I. Wang. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449*, 2025b.
- Eric Wiewiora. Potential-based shaping and Q-value initialization are equivalent. *Journal of Artificial Intelligence Research (JAIR)*, 19:205–208, 2003.
- Jianhao Yan et al. Learning to reason under off-policy guidance. *arXiv preprint arXiv:2504.14945*, 2025.
- John Yang, Kilian Lieret Zhang, et al. SWE-smith: Scaling data for software engineering agents. *arXiv preprint arXiv:2504.21798*, 2025.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

- Qiyong Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Yuanqing Yu et al. Steptool: Enhancing multi-step tool usage in llms through step-grained reinforcement learning. *arXiv preprint arXiv:2410.07745*, 2024.
- Lifan Yuan, Wendi Li, Huayu Chen, Ganqu Cui, Ning Ding, Kaiyan Zhang, Bowen Zhou, Zhiyuan Liu, and Hao Peng. Free process rewards without process labels. *arXiv preprint arXiv:2412.01981*, 2024a.
- Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024b.
- Yang Yue et al. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.
- Chenchen Zhang. From reasoning to agentic: Credit assignment in reinforcement learning for large language models. *arXiv preprint arXiv:2604.09459*, 2026.
- Chujie Zheng, Shixuan Liu, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.
- Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. minif2f: a cross-system benchmark for formal olympiad-level mathematics. *International Conference on Learning Representations (ICLR)*, 2022.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685*, 2023.
- Yifei Zhou, Andrea Zanette, Jiayi Pan, Sergey Levine, and Aviral Kumar. ArCHer: Training language model agents via hierarchical multi-turn RL. In *International Conference on Machine Learning (ICML)*, 2024.
- Xinyu Zhu et al. The surprising effectiveness of negative reinforcement in LLM reasoning. *arXiv preprint arXiv:2506.01347*, 2025. NeurIPS 2025.

A Full Experiments on Dense Potentials

The main text treats the dense progress *potential* as the reachability-gated half of the process signal (§4). This appendix gives the full experiments. The picture is consistent with the variance account: where a potential’s intermediate values are *reachable* it supplies gradient the outcome lacks and accelerates training, and where they are unreachable it is exactly vacuous. Converting the extra gradient into a stable success gain requires a bounded optimizer—an aggressive update rule can destabilize the densely shaped policy, an optimization caveat we return to in §A.3. Unless noted, we train Qwen3 [?] with our own GRPO, place any dense credit on the responsible tokens with a separate per-channel normalizer, and count efficiency in *episodes generated* so that resampling costs are visible.

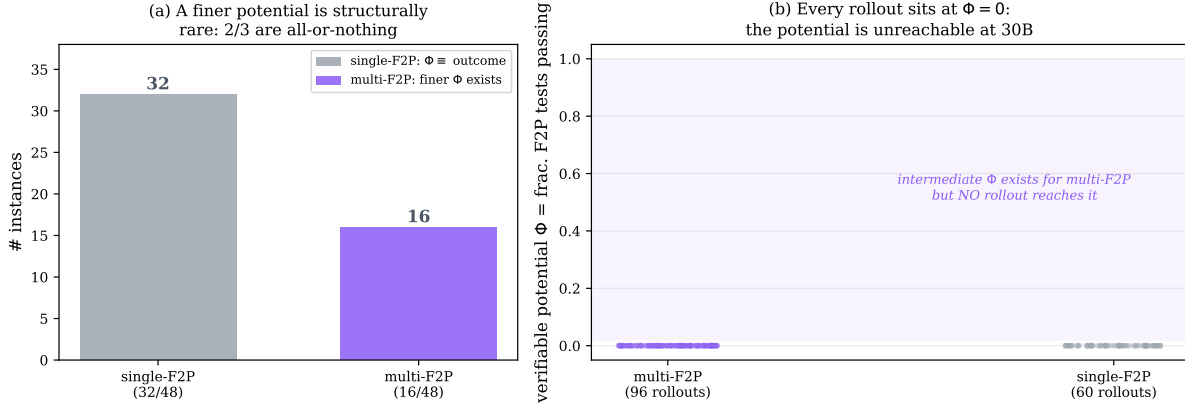


Figure 12. In software repair the finer potential is structurally rare and empirically unreachable. (a) Of 48 single-file-fix SWE-bench instances, two-thirds have a single failing test, so the test-fraction potential equals the binary outcome; only one-third admit a finer Φ . (b) For those, a 30B policy’s rollouts never realize it: all 156 rollouts score $\Phi = 0$. The band of intermediate values exists but is empty—zero within-group variance, hence zero gradient.

A.1 Reachability is measurable before training, and it gates the benefit

The condition the variance account makes operative is reachability: a potential Φ helps only where the policy realizes differing intermediate values across a group, i.e. $\text{Var}_G(\Phi) > 0$. This can be checked *before* spending training compute, from a handful of base-policy rollouts.

On SWE-bench software repair [?] the natural potential is the fraction of hidden `FAIL_TO_PASS` tests an episode makes pass. Two facts make it vacuous (Figure 12). Structurally the finer potential barely exists: two-thirds of instances have a single failing test, so Φ collapses to the binary outcome, and only a third admit any partial progress. Empirically even that third is unreachable: across 156 rollouts of a 30B policy *every* episode scores $\Phi = 0$, so $\text{Var}_G(\Phi)$ is identically zero and a Φ -based reward centers out to no gradient. The cause is not gameability but unreachability—there is no realized intermediate state for any reward to act on.

Mapping the same no-training proxy $\text{Var}_G(\Phi)$ across model capability (0.6–4B) and outcome sparsity (Figure 13a) reproduces the structure the account predicts: it rises with capability and peaks in the sparse-but-reachable corner. The measured dense-reward benefit tracks it (Figure 13b): ≈ 0 where $\text{Var}_G(\Phi) \approx 0$ (the SWE null; the too-weak small models) and large where it is high (the 4B reachable chain, dense 0.34 vs. outcome 0.01; real theorem proving, below). Benefit appears only past a reachability threshold. We could not fill the map’s interior with controlled *training* points—small-scale synthetic training is too learning-rate-sensitive, most cells degenerating to zero for both arms—so the benefit axis rests on the real-domain anchors plus one clean controlled point.

A.2 Where the potential is reachable, it removes dead updates and accelerates

When the potential’s intermediate values are reachable and point the same way as success, a dense signal is genuinely useful. We instantiate it on a family of *N-stage chained* file-operations tasks (tools `list_dir`, `read_file`, `write_file`, `delete`, `run_tests`, `submit`); success requires all *N* stages, so *N* tunes base difficulty (*N*=4 sits near 8% base success). The dense signal is a verifiable progress potential carried on a token-attached channel (a penalty when a call violates a rule—overwrite a file never read, submit an untested change—and a credit when a

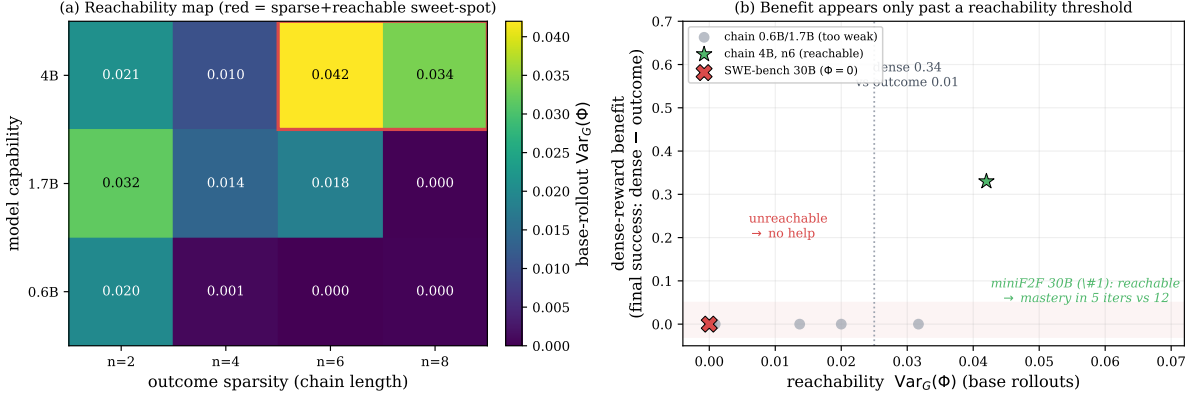


Figure 13. Reachability, measured cheaply, gates the dense-reward benefit. (a) $\text{Var}_G(\Phi)$ on base-policy rollouts (no training) over model capability $\times$ outcome sparsity: it rises with capability and peaks in the sparse-but-reachable corner. (b) The benefit of a dense reward (final-success gap, dense – outcome) against that same probe: zero where $\text{Var}_G(\Phi) \approx 0$ and large where it is high. Benefit appears only past a reachability threshold.

call discharges a pending obligation).

Dead updates. We count the fraction of iterations whose update is *dead*: an all-fail group with no reward spread (Figure 14). Outcome-only GRPO is dead on 65% of iterations. DAPO [?], built to dodge all-fail groups by resampling, reaches 54% but pays a $5.6\times$ generation tax. The aligned potential cuts dead updates to 8% at no extra sampling, by producing gradient from the very failures the outcome reduces to zero. The cure is the credit scheme, not the budget—and it is contingent on reachability: on the silent-precondition gate (§A.4), where the pivotal action is never sampled, every reward-only method is dead almost always (outcome and DAPO 100%, the potential 76%), because a discharge can fire only on behavior the policy attempts.

Speed. On held-out success against episodes generated (Figures 15, 16) outcome-only crawls, spending its budget on dead updates, then converges only at the end; the aligned potential rises immediately from the early all-fail groups. Reimplemented dense baselines (GiGPO-style step advantages [?], StepTool-style per-call rewards [?]) also outpace outcome-only—any *admissible* dense signal helps. The gain also holds on real theorems (miniF2F [?]) at 4B and 30B, but only in the multi-seed form quantified in §A.3: a single-run pilot of ours showed a dramatic $2.4\times$ speed-up that *flipped sign* when the identical configuration was re-run (rollout sampling is unseeded in the serving engine, and MoE training at this scale is bimodal), so we report only the five-seed matrix (Figure 10, Table 2): a $\sim 1.6\times$ speed-up to mastery under a matched optimizer, consistent on every seed, plus the reliability advantage. Both arms saturate—the claim is speed and reliability, not final success.

A.3 Converting the gradient to success: protocol, and the matched matrix

The extra gradient a reachable potential supplies must still be turned into task success, and here the decisive factor is the optimizer, not the potential. Early runs read this as “fragility,” but the cause was two experimental artifacts. First, an aggressive, unbounded update rule

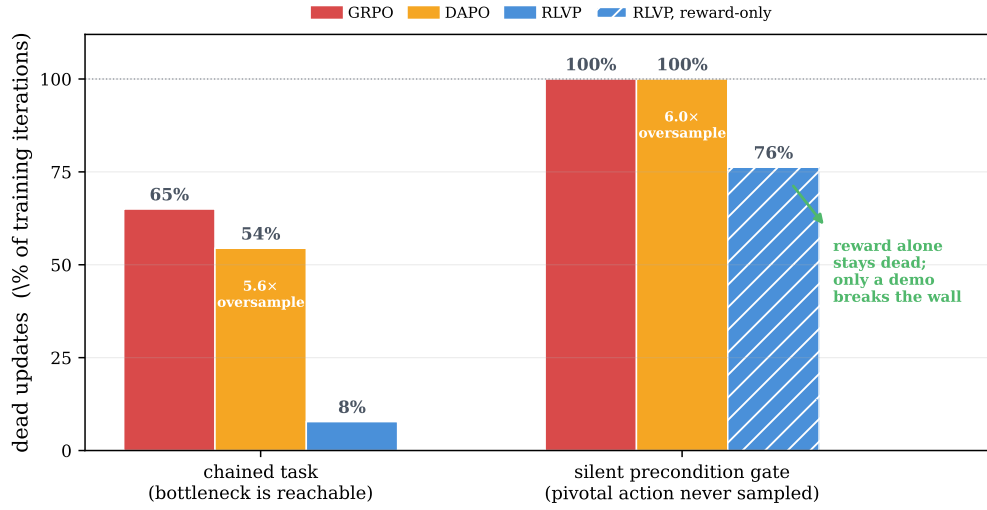


Figure 14. Dead updates across two regimes. Fraction of iterations with an all-fail, zero-spread group (Qwen3-4B, 3-seed mean). *Chained task* (bottleneck reachable): outcome-only 65%, DAPO 54% at a $5.6\times$ generation tax, the aligned potential 8%. *Silent precondition gate* (§A.4, pivotal action never sampled): outcome and DAPO dead on *every* iteration, the reward-only potential 76%; only a demonstration seed breaks the wall. The cure is a *reachable, admissible* signal, not density.

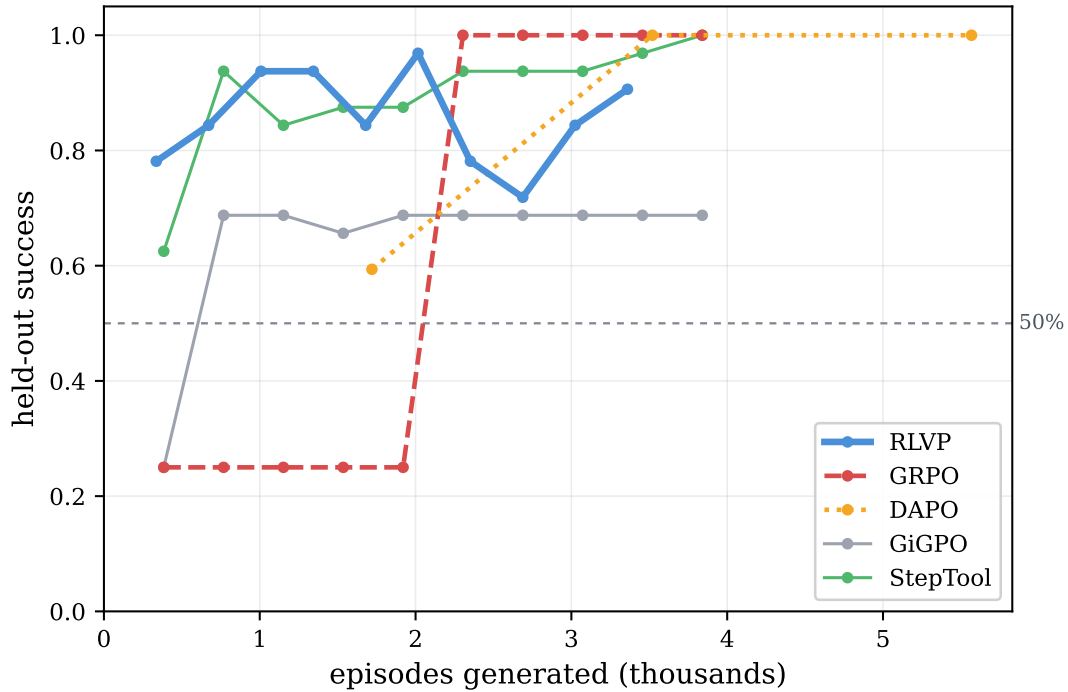


Figure 15. Sample efficiency on the calibrated hard task ($N=4$). Held-out success vs. episodes generated. Outcome-only sits near base rate for most of training; the aligned potential climbs immediately from the all-fail groups; dense baselines also outpace outcome-only. The x -axis counts episodes *generated*, including DAPO’s resampling.

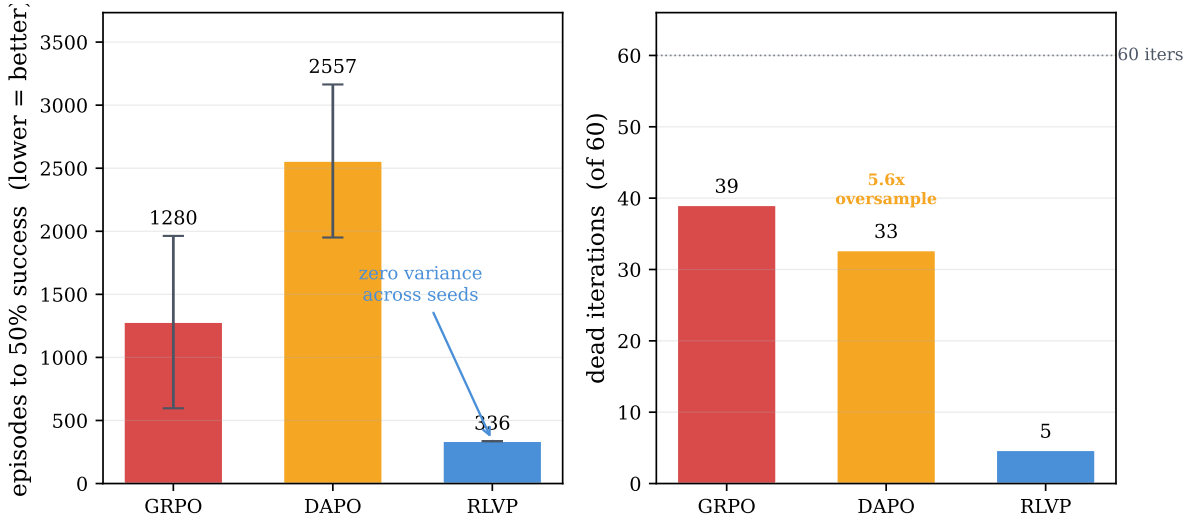


Figure 16. Efficiency over three seeds. *Left:* episodes to the success threshold (lower better)—the potential is several times faster than GRPO and DAPO. *Right:* the mechanism—GRPO and DAPO burn most iterations on dead or stalled updates, and DAPO generates 5.6 $\times$ the episodes it uses. Note the large GRPO error bar: the all-fail lottery on the outcome side, which the dense potential’s reachable within-group variance avoids.

can destabilize a densely shaped policy: at one fixed learning rate, a theorem-proving run anneals cleanly to full success (gradient norm < 1.5 , entropy decaying to zero) while an otherwise-identical sibling suffers a gradient blow-up (norm $> 10^9$), its entropy explodes, and success collapses to zero and never recovers—the same signal, the same rate, decided by optimization stability. On the chained task the opposite failure appears: the policy collapses to a zero-entropy degenerate output, so every rollout is identical, the within-group variance is zero, and the update dies. A *bounded* optimizer (we use Muon, with a divergence guard that aborts on entropy collapse or gradient spikes) removes both failure modes. Second, per-iteration training success measured on a handful of freshly sampled tasks is too noisy to read a speed-up from—it swings between extremes iteration to iteration—so we evaluate on a fixed held-out task set. The result robust to all of this—the mechanism the paper relies on—is dead-update elimination: zero dead iterations against ~ 16 of 40 for outcome-only, on *every* seed.

The matched matrix: five seeds, two scales. With the bounded-optimizer protocol fixed (aligned goal-progress potential as a token-attached discharge, Muon, a divergence guard, 30 iterations on miniF2F algebra), we ran the full matrix—aligned potential versus outcome-only, five seeds per arm, at 4B and 30B, plus a three-seed anneal ablation and a 30B AdamW outcome baseline; all magnitudes are in Table 2. Because rollout sampling in the serving engine is unseeded, “seeds” are i.i.d. draws of the whole run, so per-arm consistency is the meaningful statistic. Three facts are seed-robust. **(1) A modest, consistent speed-up** under a matched optimizer: the aligned potential reaches the threshold sooner and has higher area-under-curve on every seed-matched pair at 4B. **(2) Reliability:** it completes every run across both scales without a divergence abort, while outcome-only loses seeds to entropy collapse (4B) and gradient explosion (30B)—the dense, always-relevant gradient appears to *regularize* where the bursty sparse signal blows up under the same rate. **(3) The outcome arm’s dilemma at 30B:**

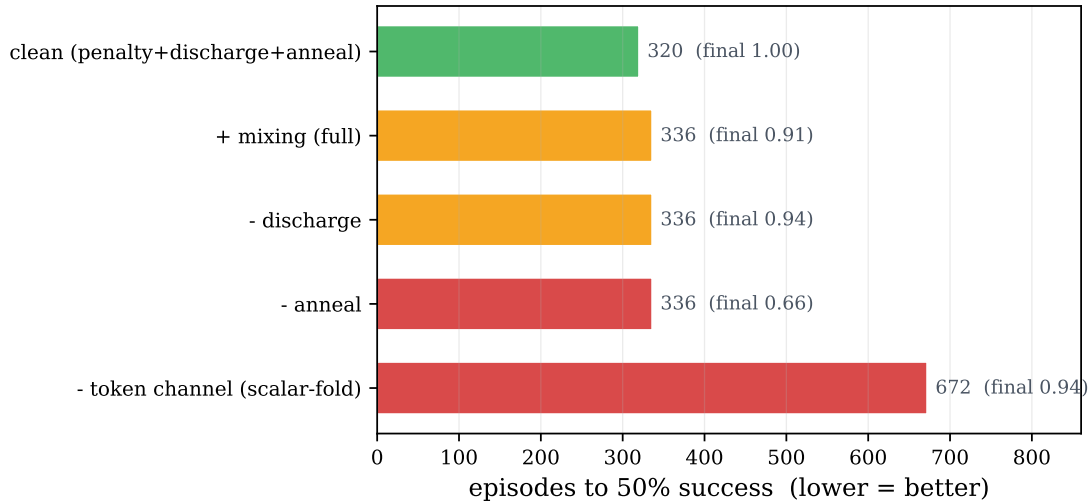


Figure 17. Component attribution for the potential on $N=4$ (episodes to threshold, converged success annotated). The token-attached credit channel is the efficiency lever (folding into a scalar return doubles the cost); annealing protects the ceiling; the clean recipe is best. A per-step length cost, useful only on very long horizons, hurts here.

outcome-only is competitive under the fast optimizer only when it survives (it diverges on most seeds), and stable under its safe optimizer only at several times the potential’s time-to-mastery. The aligned potential requires no such choice.

Anatomy of the gain. When the potential does win, ablating on $N=4$ (Figure 17) attributes the speed to the token-attached channel: folding the same rule rewards into the scalar return doubles the episodes to threshold. Annealing the shaping off after saturation protects the final ceiling (unrelaxed pressure drifts the policy toward over-compliant, bloated episodes—a mild inaction-trap tendency). The discharge credit is the positive signal that escapes all-fail groups; removing it leaves more dead iterations and a lower ceiling. Finally, replacing every hand-written rule with three universal meta-rules derived only from per-tool category tags (observe/mutate/verify/terminal) and the environment’s error codes recovers the hand-engineered efficiency almost exactly: in well-structured environments the specification cost can be paid by metadata the tool schemas already expose.

A.4 Reachability governs the ceiling, not just the speed

Sample efficiency is speed to a ceiling the chained tasks eventually reach. To ask whether a process reward can raise the *ceiling*, we build a task that is hard to *discover* rather than hard to compute: a *silent precondition gate*, where to write a protected file the agent must first read an access-control file and request access; skip either and the write silently no-ops (reports success, does nothing). This is the bank-authentication problem of the introduction in controlled form—a required precondition an agent cannot stumble onto by trial and error and must be told—and it is where a purely reward-driven online learner would stall indefinitely. Calibration confirms the trap is total—the base model never reads the access file even when the rule is in its prompt. Then (Figure 18) *every* reward-only method (outcome, DAPO, the clean potential) fails to zero, because the discharge for reading the access file can only reward that behavior if the policy

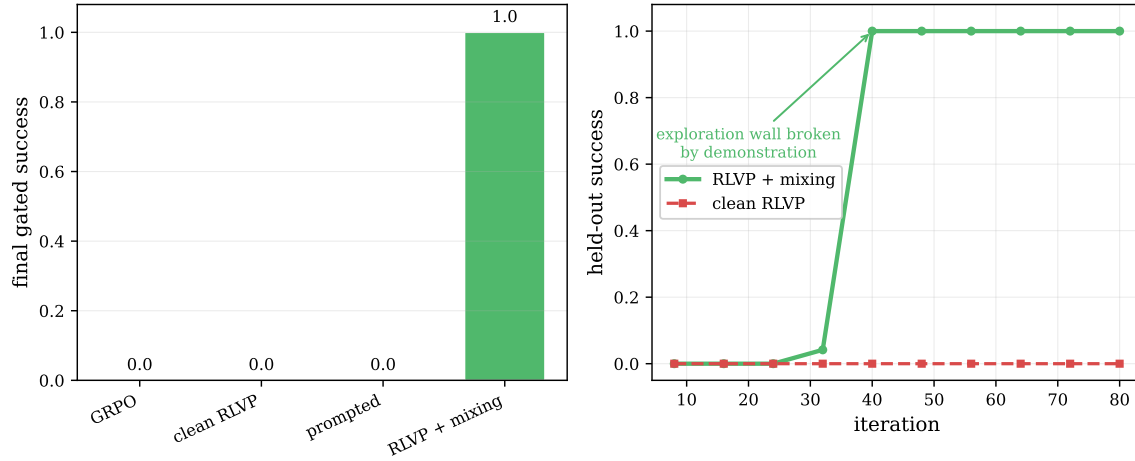


Figure 18. A non-saturating ceiling test, and the discovery wall. *Left:* on the silent-gate task every reward-only method (outcome, DAPO, clean potential, even rules-in-prompt) converges to zero—the precondition is never sampled, so the discharge never fires. *Right:* a single synthesized demonstration through the process channel breaks the wall to perfect held-out success, a sharp phase transition.

samples it, and it never does. A single rule-synthesized demonstration injected through the process channel breaks the wall to perfect, generalizing success. When the required behavior is never sampled, no on-policy reward can induce it; imitation must seed what reward then grows—the same reachability wall the SWE probe hit from the other side, and the R1-Zero (discoverable) versus R1 (demonstration cold-start) distinction in miniature.

A.5 Learning where the outcome is dead: real bug-fixing at zero success

The variance account predicts that when a group is all-failing—outcome variance zero, outcome gradient dead—a process signal is the *only* thing that can move the policy. We check this directly on real software repair, a domain where the path-vs-outcome gap is well documented: in a controlled industry study, AI coding agents reach high functional correctness yet score far lower—and almost at random—on reliability, maintainability, and not breaking previously working functionality [?], the engineering discipline that decides whether a patch is deployable. Here an 8B agent is far below the task’s reach, which makes it an ideal test of the dead-gradient regime. We train on SWE-smith [?] instances (a repo container, a hidden FAIL_TO_PASS test as the oracle), with the agent issuing bash commands to find and fix the bug; the process channel is the coding-discipline signal of §3 (a discharge for running tests and making a verifiable edit, a penalty for re-running a failed command or editing without testing). At this scale the agent solves essentially *zero* instances under either signal, so outcome-only RL trains on an all-fail distribution throughout—the cleanest possible test of the dead-gradient regime. This is a *different* signal than the progress *potential* of §4: on this same software-repair domain the test-fraction potential is vacuous (Figure 9, left), because zero tests ever pass and so no intermediate value is reached. The penalized and discharged behaviors here—running a test, editing a file, re-running a failed command—are always reachable, so the penalty channel carries within-group variance exactly where the potential cannot. The two software-repair results are consistent: the reachability gate closes on the potential and stays open on the penalty.

The result is a sharp dissociation (Figure 19, two seeds). Outcome-only leaves the trajectory

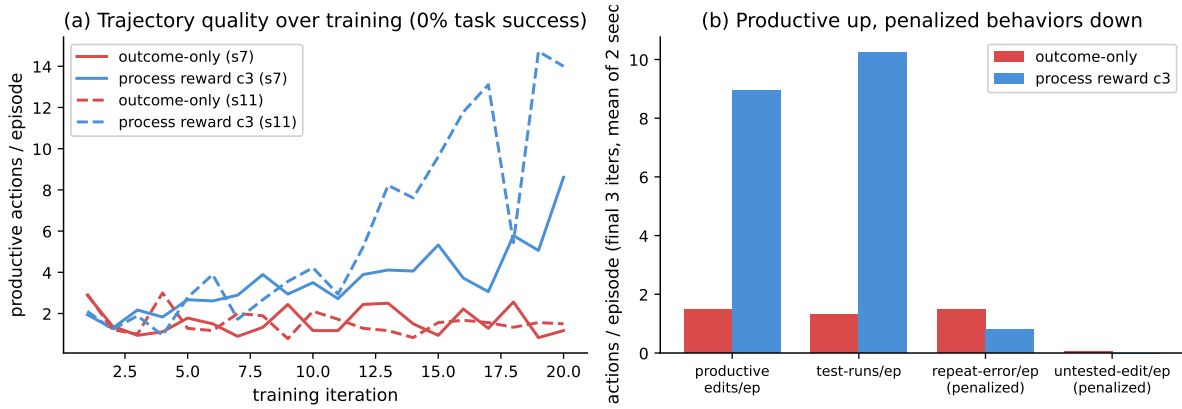


Figure 19. The process reward learns where the outcome is dead (SWE-smith, 8B, $\approx 0\%$ solve). (a) Productive actions per episode over training, two seeds: outcome-only is flat (the all-fail gradient is dead), while the process reward rises steadily. (b) Final-3 trajectory quality (mean of two seeds): productive edits and test-runs increase several-fold under the process reward, while penalized behaviors (repeat-errors, untested edits) fall. Task success is zero for both arms throughout—this is a behavioral/mechanistic result, not a success result.

flat: productive actions per episode sit at ≈ 1.5 from the first iteration to the last, essentially identical across seeds (1.5 and 1.5)—the dead gradient changes nothing. The process reward instead drives a steadily *rising* trajectory: productive actions climb over training to 6.5 and 11.4 (final-3, the two seeds) and test-runs to 7.6 and 12.9, a four-to-tenfold increase, while the penalized behaviors fall (repeat-errors and untested edits toward zero). The agent learns to investigate, test, and edit like a disciplined engineer—from a signal available on every failed episode—exactly where the outcome reward is inert.

Two honest caveats scope the claim. First, the discharge *rewards* test-running and productive edits, so part of their rise is by construction; the non-circular evidence is that the *penalized* behaviors (which the discharge does not touch) also fall, and that the outcome baseline is perfectly flat—the effect is the process gradient, not the task. Second, this improved discipline did *not* convert to task success in our budget (both arms remain at zero): 8B is simply too weak to close these bugs. The claim is therefore mechanistic and behavioral—the process channel produces learning, and better verifiable engineering discipline, precisely in the all-fail regime where outcome-only RL has no gradient at all—and a plausible precursor to success at capability, not a demonstration of success itself.

B Boundaries: When the Penalized Target Is Not Verifiable

Design rule 4 (§3.3) requires the penalized (and discharged) target to be reachable and *un-gameable*. This appendix gives the experiments behind that rule: what happens when a penalty’s target is misaligned with the outcome, and when it is a *learned* proxy rather than a verifiable predicate.

B.1 An un-gameability sweep: which signals survive

We construct five process signals for Lean theorem proving on miniF2F [?], spanning aligned to misaligned, pre-register each one’s cheapest gaming policy, and train Qwen3-30B-A3B [?]

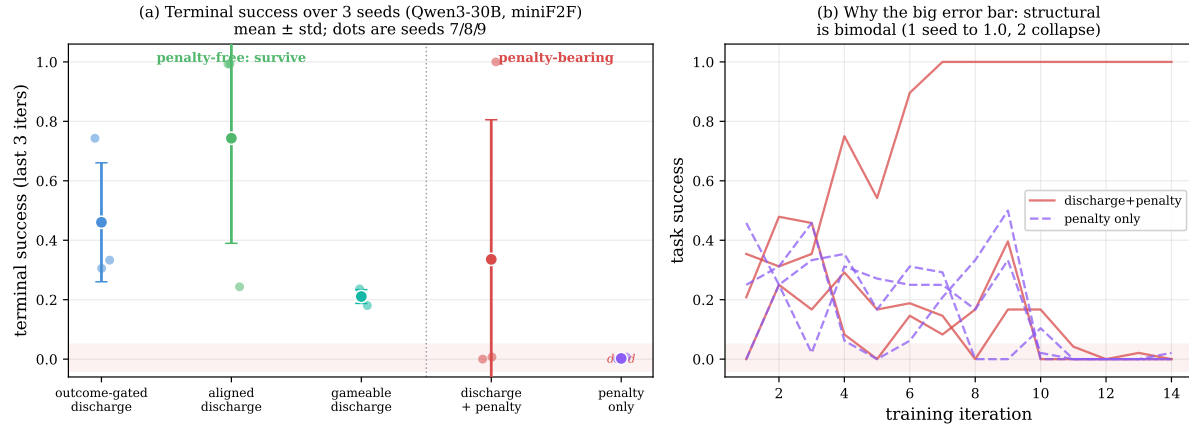


Figure 20. Which signals survive (Qwen3-30B, miniF2F, 3 seeds). (a) Terminal success (last-3-iteration mean) for five signals, mean $\pm$ std with seed dots. The three penalty-free signals sit well above the dead zone on every seed; the pure penalty is reliably dead; the discharge+penalty arm has by far the largest error bar. (b) That arm is *bimodal*—collapsing on two seeds, rescued to 1.0 on one—whereas the pure penalty collapses on all three.

with each (three seeds, everything else fixed; Figure 20). Survival tracks admissibility. The *penalty-free* signals reliably survive on every seed: a gameable “any-valid-tactic” credit, an aligned goal-progress discharge, and that credit *gated on the eventual outcome* (un-gameable by construction, the most consistent survivor). The *pure penalty*—an errored-tactic penalty with no discharge and no outcome reward—reliably **collapses to essentially zero on every seed**: its cheapest maximizer, avoid errors by not attempting, is precisely the *inaction trap* of design rule 2, and with no positive signal to escape it the policy dies every time. This is the paper’s cleanest evidence that a lone penalty cannot drive learning. The revealing case is *penalty-plus-discharge*: across three seeds it is bimodal (collapses on two, rescued to perfect on the third), the largest variance of any arm—adding a discharge to a misaligned penalty makes survival *possible* but not reliable. The robust reading: penalty-free progress signals survive, a pure penalty reliably collapses, and outcome-gating is the most consistent survivor. Precise magnitudes remain noisy at 30B; we report the survival pattern.

B.2 Task intent is not a verifiable rule

The hardest case for a penalty is a task whose difficulty is not procedural. On τ -bench-style airline customer service [?] the reward turns on *semantic* policy adherence (authorization, refund eligibility, confirmation) rather than structural ordering, so no verifiable rule is finer than the outcome: what is missing is intent, which is the reward itself. Three tiers make this concrete (Figure 21). *Generic* structural rules—the read-before-write hygiene that works on chained tasks—are now orthogonal to the reward and actively *harm*: the cheapest way never to violate “confirm before calling” is to not place the risky calls at all, and the policy collapses into the inaction trap within a handful of iterations. *Policy-derived procedural* rules (fetch the user id and look up the reservation before modifying it), outcome-instrumental by construction and paired with early annealing, remove the harm, because a correct modification requires the looked-up state so the discharge pulls toward the productive workflow. *Verifiable semantic* rules (do not modify a basic-economy reservation; do not use a payment method absent from the profile) prune failure-bound actions and lift the best iterations nearly to outcome-only’s peak.

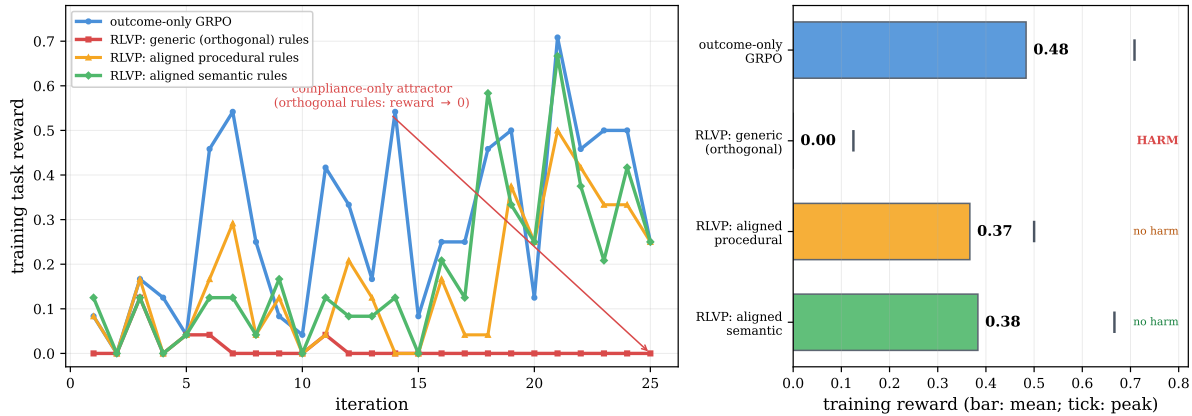


Figure 21. The coverage gradient on τ -bench airline. *Left:* training reward for four signals. Outcome-only learns the benchmark; generic rules orthogonal to the reward collapse into the inaction trap; policy-derived procedural and verifiable-semantic rules (with early annealing) do not. *Right:* mean reward per tier (bars) with peak (ticks). Misaligned rules harm; aligned rules remove the harm and nearly reach outcome-only’s peak, but neither beats it. The residual gap is intent.

Is rule-following instrumental to the outcome?

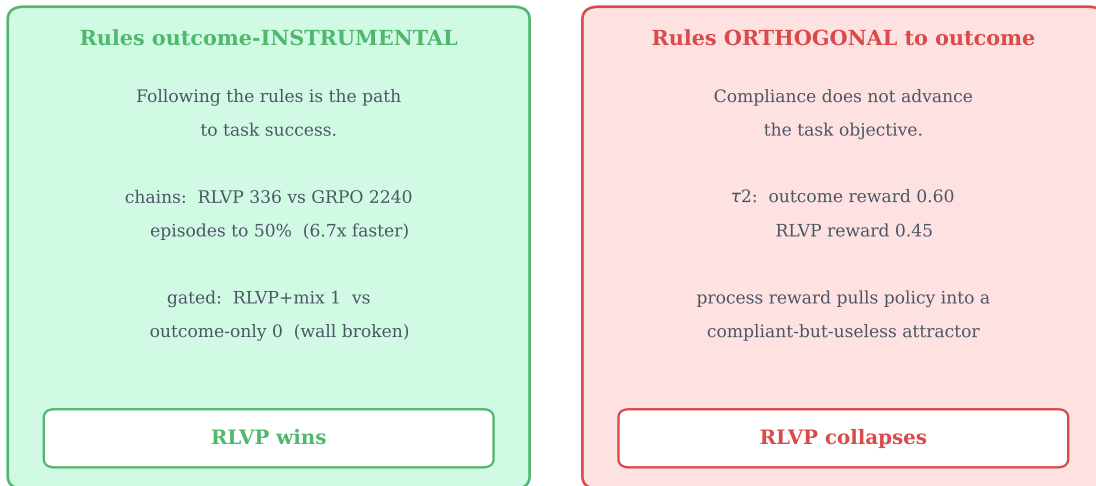


Figure 22. The boundary on one axis: are the rules outcome-instrumental? When the verifiable rules are aligned with what success requires (chained and gated tasks: the workflow *is* the task), the process channel converts compliance into large outcome gains. When orthogonal to the reward (τ -bench, generic rules), the same machinery optimizes compliance into a degenerate policy.

But neither aligned tier *beats* outcome-only on average (Figure 22): verifiable rules express policy *validity*, while the residual difficulty is *which* permissible change this user wants—and a rule encoding that would just be the task specification.

B.3 A learned critic relocates the hack

If verifiable rules cannot reach intent, why not let the policy judge itself, in the lineage of self-rewarding LMs and RLAIIF [????]? We use the critic as the *same model* as the policy (no larger judge), so any signal is genuine on-policy self-reflection. Across two axes—whether a

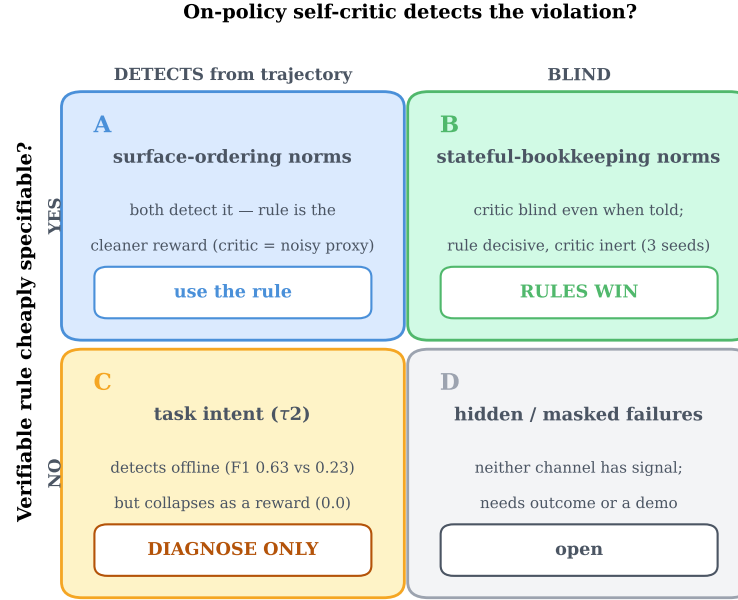


Figure 23. When a verifiable rule beats a same-model self-critic, and when neither trains. Axes: whether a verifiable rule is cheaply specifiable, and whether the on-policy critic can detect the violation. The learned critic is a poor training reward in every quadrant; its only robust use is offline diagnosis at the intent frontier.

verifiable rule is cheaply specifiable, and whether the on-policy critic can detect the violation (Figure 23)—the learned critic is a poor training reward in every quadrant. As a detector, blind self-critique flags surface-evident violations but is essentially blind to stateful-bookkeeping norms (did it read *this* file before overwriting it) even when handed the rules, and the blind spot is structural (per-rule recall stays near zero as the critic scales). As a reward in the all-fail regime (Table 3), a deterministic rule with identical credit structure is decisive on every seed while the self-critic—with *perfect recall* against the rule oracle—is inert; a *frozen* critic fails identically, isolating the cause as the critic’s $\sim 6\%$ false-positive rate, not its non-stationarity. At the intent frontier (Table 4) the self-critic is a useful *offline* detector yet collapses as a training reward. Defining the penalized target by a learned proxy relocates the credit-assignment problem into the proxy, which the policy then games—the mechanism behind design rule 4. Verifiability is what makes the dense channel safe to optimize.

C Setting-by-Setting Summary

Table 5 collects the five settings studied across the paper against the three properties the variance account cares about—does a verifiable signal finer than the outcome exist, is it un-gameable, and are its intermediate values reachable—together with the observed outcome. Help appears only when a reachable, un-gameable signal is available; a penalty on always-sampled bad actions satisfies this on every row where it applies, while a progress potential does so only where partial progress is reachable.

Table 3. A deterministic rule beats a same-model self-critic as a dense reward, in the all-fail regime (consumer-service domain, 1.7B, three seeds, late-training mean $\pm$ std). Rule and critics share the identical penalty-only credit structure, and the critic additionally has *perfect recall* against the rule oracle. The rule is decisive every seed, while the critics are inert. The *frozen* (stationary) critic fails like the live one, isolating the cause as the critic’s $\sim 6\%$ false-positive imprecision rather than non-stationarity.

process reward	detection (P/R)	violations/episode	task success
rule penalty (deterministic)	1.00/1.00	0.38 ± 0.44	0.33 ± 0.47
self-critic, live (co-evolving)	0.94/1.00	0.94 ± 0.05	0.00
self-critic, frozen (stationary)	0.94/1.00	0.93 ± 0.02	0.00

Table 4. Self-critique is a usable intent detector but a failed intent reward (τ -bench airline, Qwen3-4B). *Left*: as a failure predictor on rule-clean (intent) failures the self-critic far exceeds the verifiable semantic rules (3 rollout seeds). *Right*: as the dense training reward the same self-critique collapses, while outcome-only and the rules both learn (training reward, early $\rightarrow$ late, 20 iterations).

<i>offline: failure prediction</i>		<i>trained as reward</i>	
channel	F1	channel	reward (early $\rightarrow$ late)
semantic rules	0.23 ± 0.06	outcome-only	$0.09 \rightarrow \mathbf{0.50}$
blind self-critique	0.63 ± 0.02	semantic rules	$0.10 \rightarrow 0.35$
		blind self-critique	$0.01 \rightarrow \mathbf{0.00}$

Table 5. The five settings studied, summarized against the variance account. Each row is a domain. The three middle columns are the conditions the account cares about: does a verifiable signal finer than the outcome exist, is it un-gameable, and are its intermediate values reachable. “ $\checkmark/\times$ ” marks the condition that decides the row. A dense signal helps only when all three hold; this table organizes the settings rather than predicting them.

Setting	Domain model	/	Finer Φ ?	Un-game?	Reach?	Verdict	Observed
Theorem proving (progress)	Lean miniF2F 30B, synth. 4B		$\checkmark$	$\checkmark$	$\checkmark$	help	dead updates $\rightarrow$ 0, dense gradient
System admin (harm)	TerminalBench 4B		$\checkmark$	$\checkmark$	$\checkmark$	help (harm axis)	$\sim 6\times$ fewer violations, equal success
Lean signal sweep (controlled)	miniF2F 30B		varies	varies	$\checkmark$	per arm	pure penalty dies, penalty-free survive
Customer service (intent)	τ -bench airline 4B		$\times$	—	—	no help	rules match, never beat outcome
Software repair	SWE-bench 30B		1/3 only	—	$\times$	no help	0% solve, all roll-outs $\Phi=0$